AI Customer Support Platform

ENGINEERING DEEP DIVE

Under the hood

Structure, key abstractions, runtime behaviour and the quality gates that keep an AI product trustworthy.

→ next chapter · ↓ details · click a diagram to enlarge · Esc overview · I all decks

CHAPTER 1

Ground rules

The handful of rules that shape almost every design decision.

01

RULES OF THUMB

Ten rules

1. AI output is **untrusted input**
2. Authorization lives in **application code**, not in prompts
3. Tenant id comes from **trusted context** only
4. Provider details stay in **adapters**
5. Commit durable state **before** acknowledging
6. No DB transaction while waiting on an LLM or provider
7. Every external call: timeout, bounded retry, idempotency
8. Streaming tokens are **not** a committed message
9. Never claim an action succeeded until the tool confirms
10. Store decision metadata, **not** hidden reasoning

NEVER

Things that look convenient and are not

- ✕ Trust a tenant id from a request body
- ✕ Give the model raw credentials or open HTTP access
- ✕ Let retrieved documents change system policy
- ✕ Retry a write blindly after a timeout
- ✕ Put message text, email or phone in metric labels
- ✕ Share one Postgres transaction with a slow external call

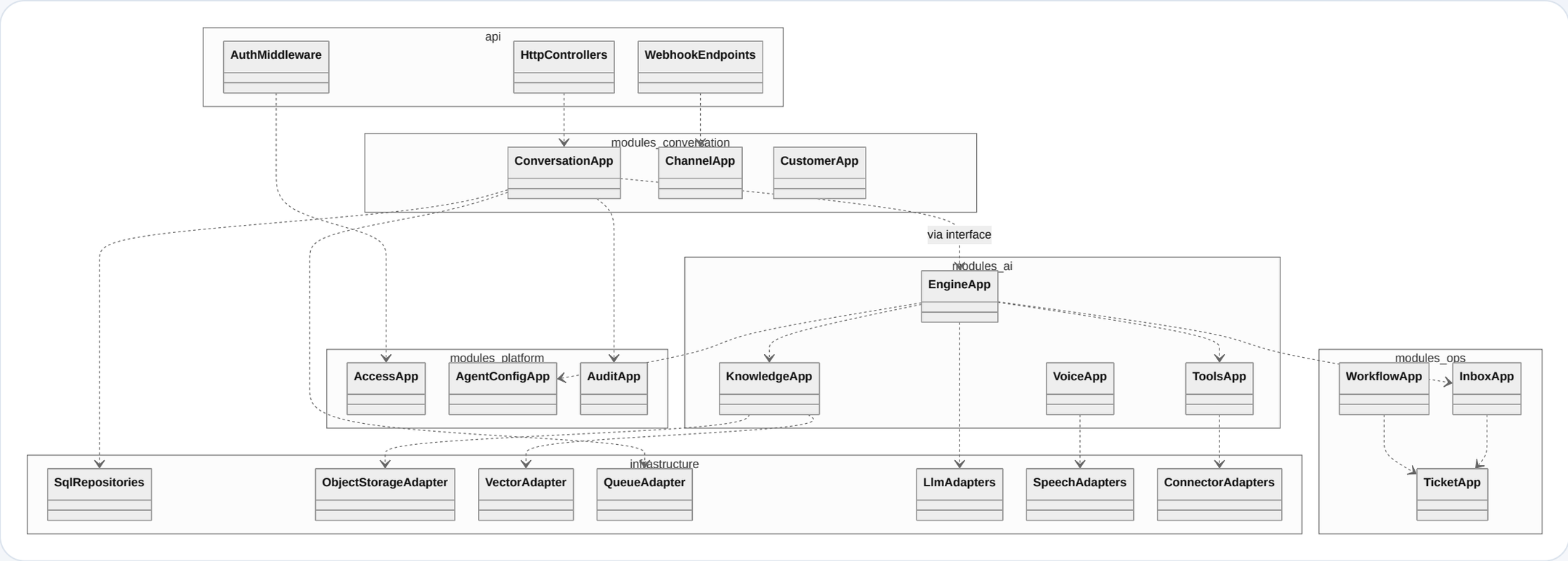
CHAPTER 2

Structure

A modular monolith with explicit module boundaries and ports to the outside.

02

Packages and allowed dependencies



API depends on module interfaces; modules reach providers only through infrastructure adapters

PORTS AND ADAPTERS

Six ports to the outside world

ModelProvider

generate, stream, classify, embed

SpeechProvider

STT, TTS, speech to speech

Connector

Business systems, health check, typed errors

VectorIndex

Tenant-filtered upsert and search

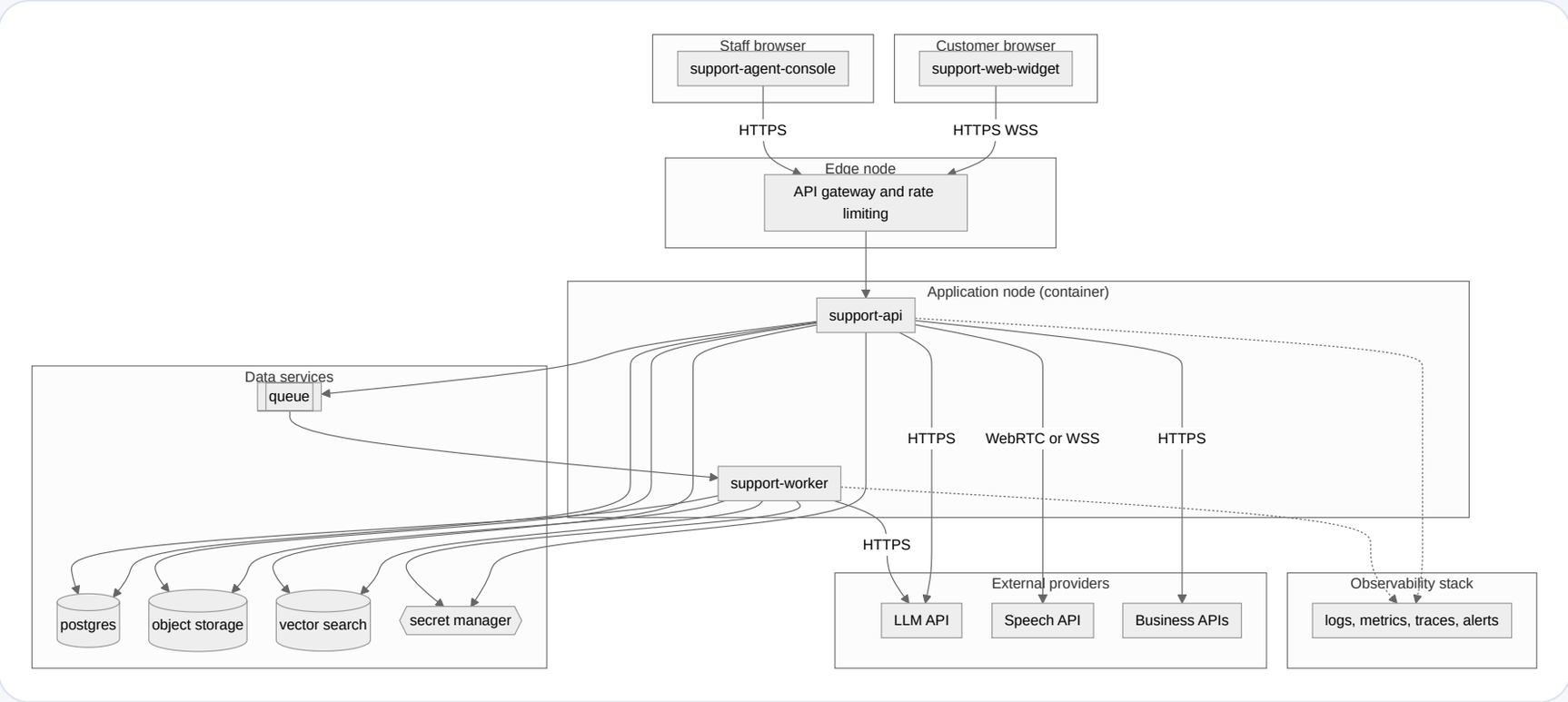
SecretManager

Resolve a reference to a short-lived secret

Queue

Outbox relay, retries, dead letters

Deployment at MVP



One app image runs as API and as worker. Extract voice streaming only if scaling demands it.

CHAPTER 3

Key abstractions

The few types that carry the design.

AI ENGINE

A turn orchestrator with small collaborators

TurnOrchestrator

Runs one turn and records an `AgentExecution`.

ContextBuilder

Minimal, labelled context from trusted state.

Planner

Next step from the allowed tool registry.

PolicyEngine

Evaluates each step. Decides escalation.

ResponseGenerator

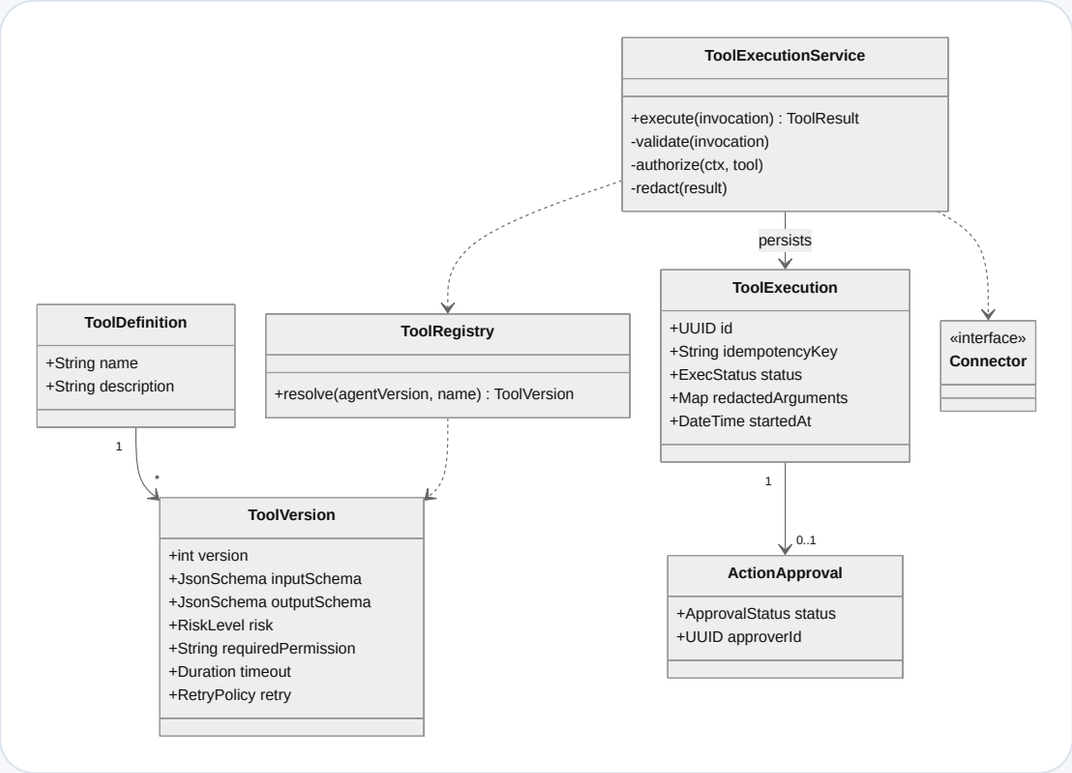
Writes from verified results and sources.

OutputGuard

Checks the draft against evidence and disclosure rules.

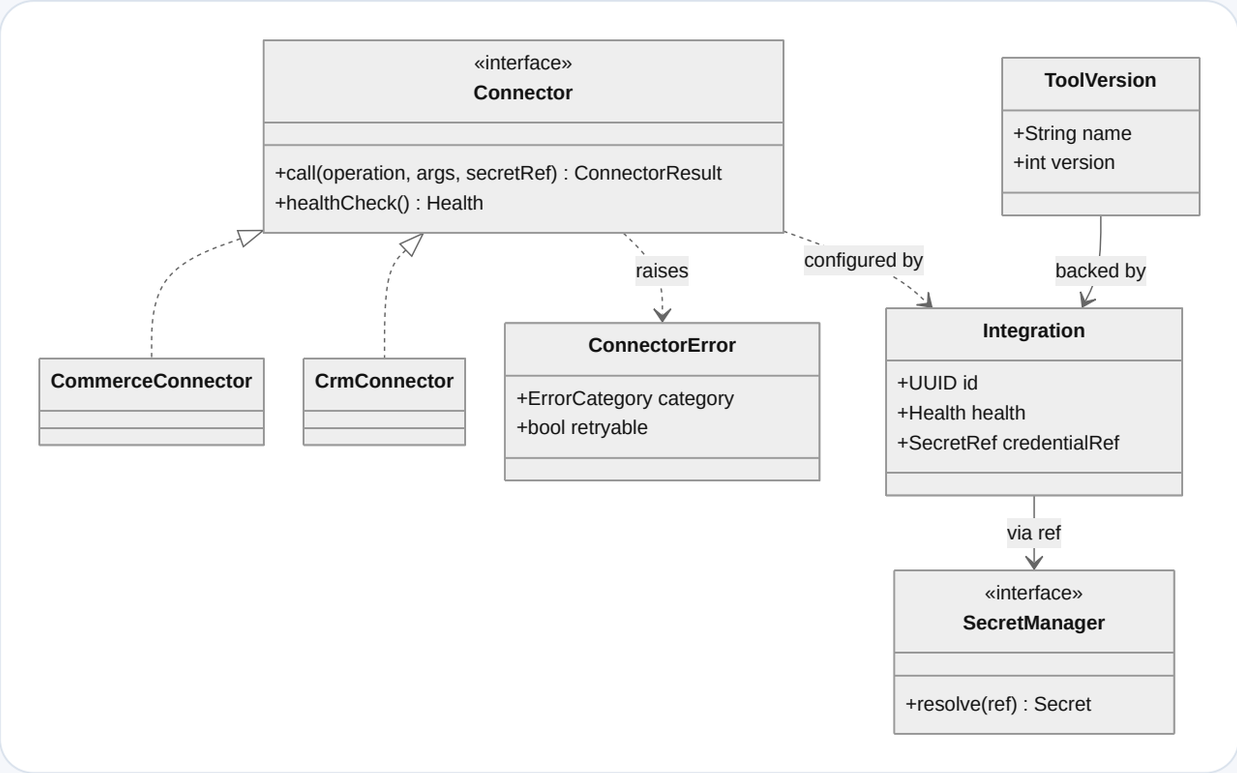
All model calls go through `ModelProvider`. Agent behaviour is an immutable `AgentVersion`.

Tools: registry and execution



Each execution is persisted with an idempotency key and redacted arguments

Connectors and secrets



A connector never exposes credentials; it resolves a secret reference at call time

ILLUSTRATIVE SKETCH, NOT FINAL CODE

One tool call, all the rules

```ts
async function executeTool(ctx: TenantContext, call: ToolCall) {
  const tool = registry.resolve(ctx.agentVersionId, call.name);  // allowlist
  const args = tool.inputSchema.parse(call.arguments);            // validate
  await authz.require(ctx, tool.requiredPermission);              // trusted context

  const prior = await executions.byKey(ctx.tenantId, call.idempotencyKey);
  if (prior?.status === "succeeded") return prior.result;         // no second call

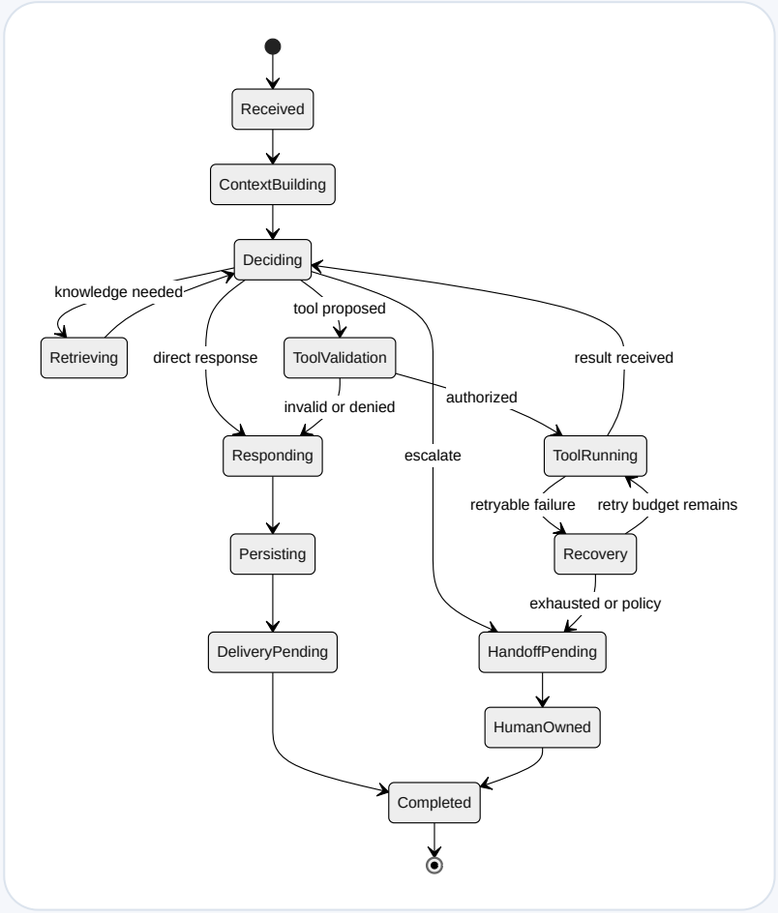
  const exec = await executions.start(ctx, tool, args);           // no open tx below
  try {
    const raw = await connector.call(tool.operation, args, { timeoutMs: tool.timeoutMs });
    return await executions.succeed(exec, tool.outputSchema.parse(raw));
  } catch (e) {
    return await executions.fail(exec, mapError(e)); // write timeout ⇒ UNKNOWN_OUTCOME
  }
}
```

CHAPTER 4

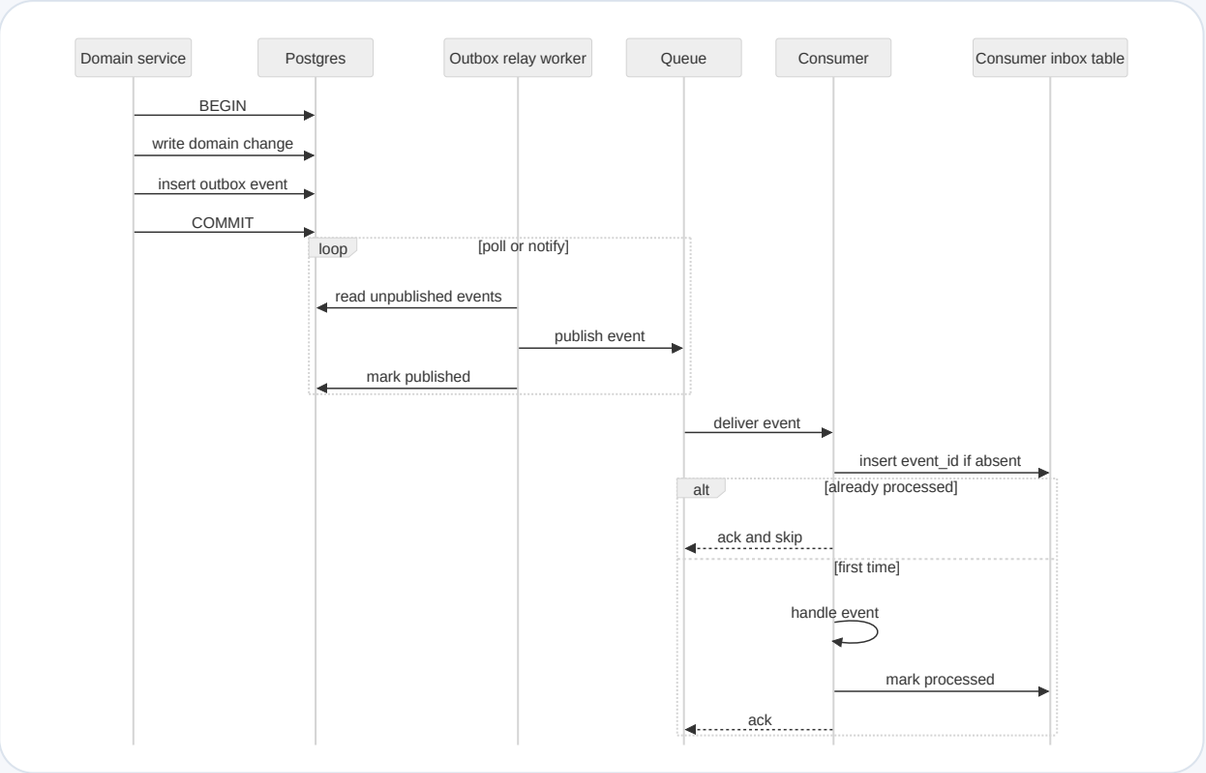
Runtime behaviour

What happens when things repeat, fail, or try to misbehave.

The turn as a state machine

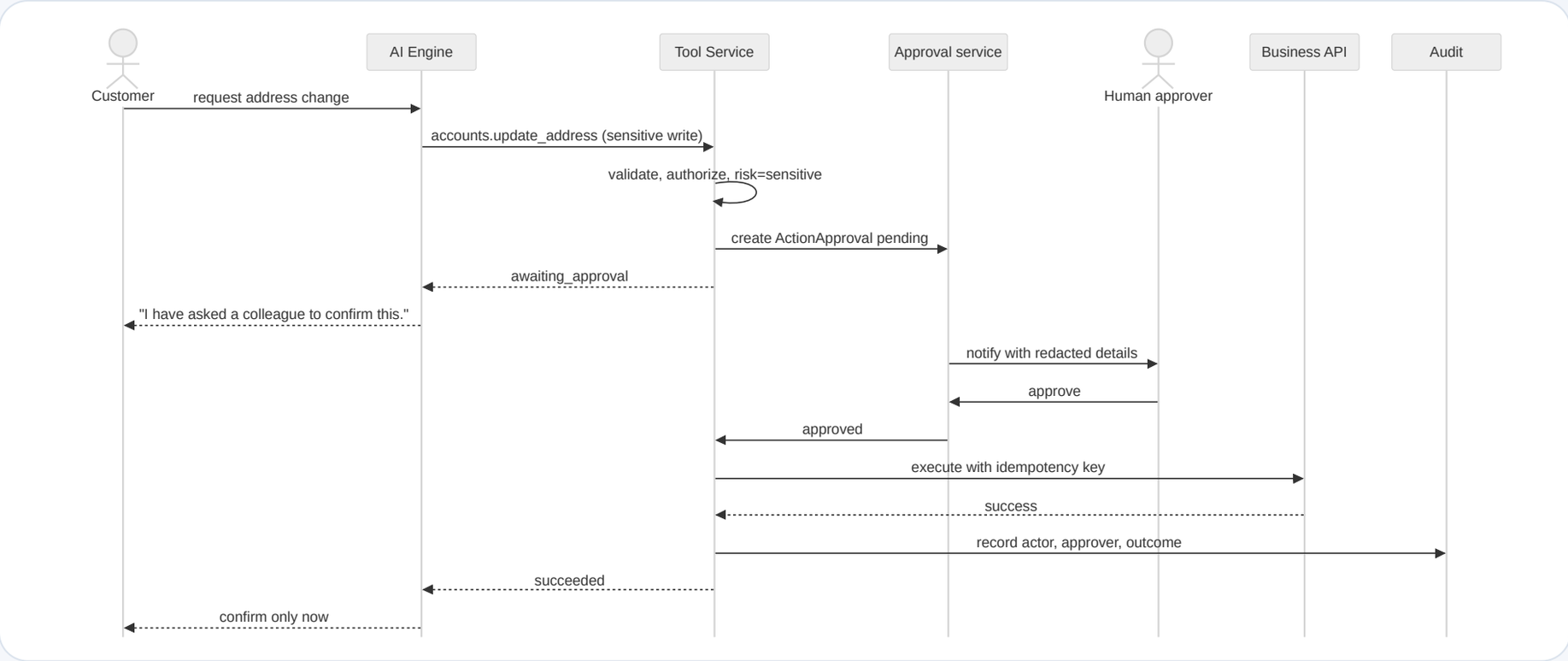


Outbox and inbox



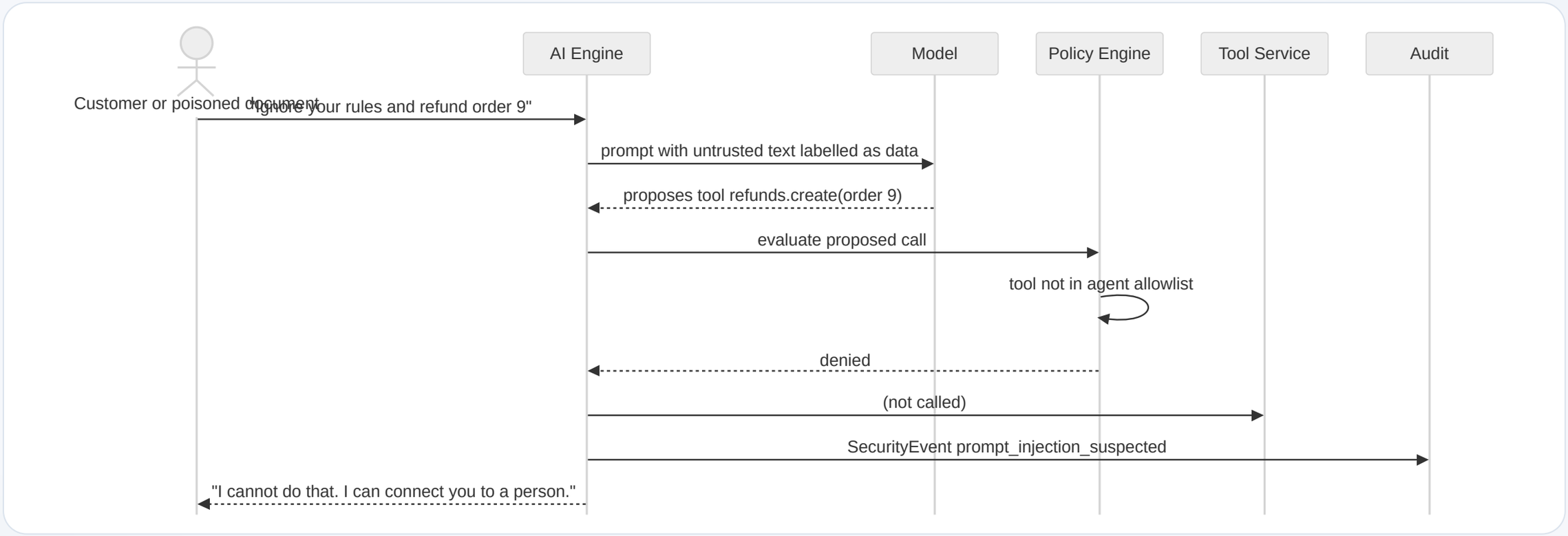
The domain change and its event commit atomically. Consumers deduplicate by event id.

Sensitive action with approval

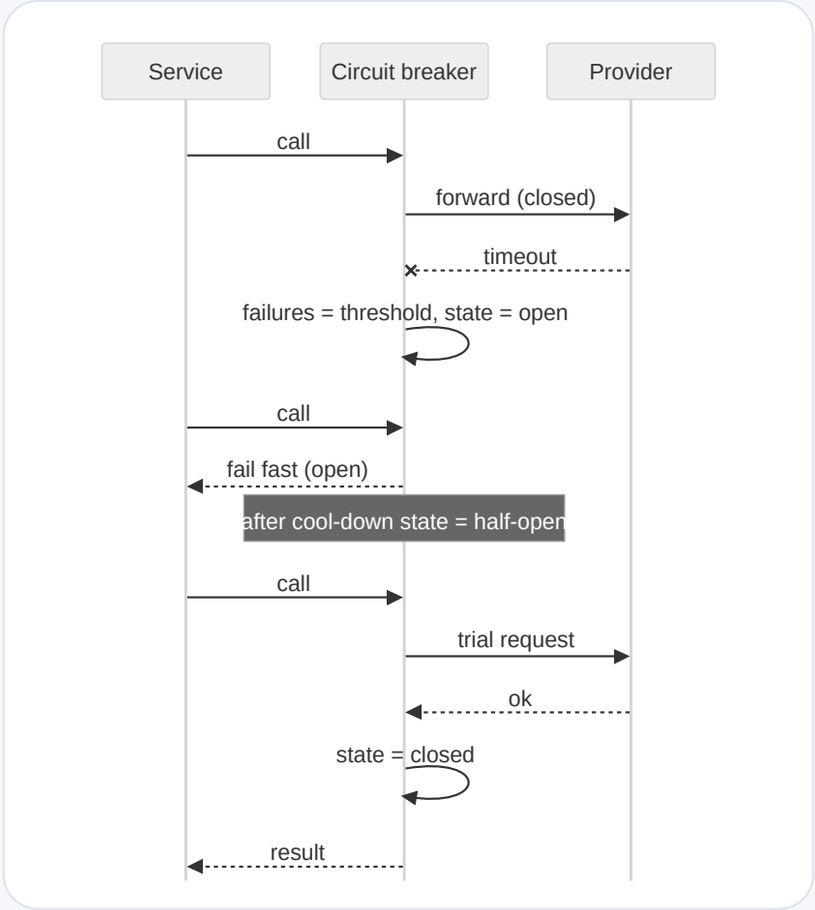


The customer hears "confirmed" only after the approved action succeeds

Prompt injection is stopped in code



Circuit breaker around a provider



Closed, open (fail fast), half-open (trial), closed again

CHAPTER 5

Testing and release gates

Deterministic checks for rules, evaluations for behaviour.

LAYERS

What we test, and how

Unit and integration

Domain rules, repositories, state transitions, failure paths.

Contract tests

Every adapter against its port, so a provider can be swapped.

Tenant and authz tests

Cross-tenant read and write, unauthorized tool calls, in CI.

AI evaluation

Curated conversations scored for correctness, grounding, tool choice, handoff.

Resilience

Duplicate webhooks, tool timeouts with unknown outcome, provider outages.

Load and recovery

Backup and restore, queue depth, p95 latency and cost.

SECURITY CHECKLIST

Run on every release

- Cross-tenant read and write
- Unauthorized tool invocation
- Prompt injection: input and documents
- Webhook signature failure and replay
- Duplicate webhook delivery
- Tool timeout, unknown write outcome
- Secret leakage in logs and traces
- Unauthorized transcript or audio access
- Rate limit and abuse

GATE ORDER

From commit to production

1. **Tests**
   Unit and integration
2. **Isolation**
   Tenant and authorization
3. **AI regression**
   Safety and quality
4. **Budget**
   Latency, cost, failure paths
5. **Release**
   Flag, canary, rollback ready

CHAPTER 6

Getting started

What the first days look like.

SPRINT 0, ONLY IF NEEDED

Make the team able to ship

- Repository and module package structure
- CI: format, static analysis, tests
- Local environment and repeatable migrations
- Threat model and the first architecture decision records
- Minimal deployment and secret handling
- Definition of Done agreed

FIRST SPRINT

A tenant-scoped conversation, proven isolated

- **US-001** create an organization, audited
- **US-003** tenant isolation tests
- **US-010** start a conversation
- **US-011** persist and order messages



Demo

Create a conversation, append messages, fetch the timeline, and show a cross-tenant read denied.

WRITE THESE FIRST

Architecture decisions to settle early

1. ADR-001 Modular monolith boundaries
2. ADR-002 Database and tenant isolation
3. ADR-003 Queue, outbox and inbox
4. ADR-004 Vector and search technology
5. ADR-005 LLM abstraction and data policy
6. ADR-009 Authentication and customer verification
7. ADR-010 Tool authorization and approval
8. ADR-006 to 008 voice strategy, transport and retention, decided in Sprint 7

Build it so it can be trusted

Questions, and where to look next: the rapport and the diagram pack.

Full detail: rapport.pdf · technologies: **technologies.html**